

מבוא לתקשורת מחשבים

הרצאה 1 – First Principles

Requirements and building blocks

תקשורת זו העברת אינפורמציה במרחב. המרחב הוא גם מרחב הזמן - למשל שמירה בזיכרון (המידע נשמר אבל הזמן עובר).
אינפורמציה, לפי שאנון, זו מחרוזת של ביטים. מחרוזת זו יכולה להיות ממורת אח"כ לקובץ jpg שמתורגם לתמונה.
פרוטוקול זו דרך לפרש מחרוזת של ביטים.
המטרה של רשתות היא לספק **חיבוריות connectivity** - להגיע מישות אחת לישות אחרת ברשת.

דרישות מהרשת

1. **משתמשים** – המשתמשים צריכים ששירותי התקשורת יהיו אמינים, שההודעות יעברו בלי שגיאה (כלומר אין ביט שהתהפך בדרך) ושההודעה תגיע ליעד.
2. **מתכננים** – המתכננים דורשים שהרשת תהיה בעלת יעילות גבוהה ועלות נמוכה.
3. **אופרטורים** – האופרטורים הם אלו שמפעילים את הרשת, כמו ספקי האינטרנט (בזק). חשוב להם שיוכלו לתפעל את הרשת בנוחות, לחייב את הלקוחות בקלות ולתקן תקלות בקלות.

ממה בנויה רשת? Building blocks

אנחנו נדבר על רשתות במונחים של גרפים - צמתים וקשתות.

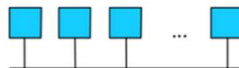
1. **צמתים** – צמתים הם תחנות בדרך, יכולים להיות מחשבים, טלפונים וכו'. יש צמתי קצה (שמייצרות ומקבלות אינפורמציה) ויש תחנות ביניים שרק מעבירות מידע כמו ראטרים וסוויצ'ים.
2. **קשתות** – למשל twisted pair, קבל קואקסיאלי, סיב אופטי וכו'.

סוגי תקשורת

1. **Point to point** – שתי תחנות עם ערוץ תקשורת ביניהן.



2. **Multiple access** – מס' תחנות שמחוברות יחד על קו אחד. כל התחנות יכולות לשמוע את המידע. מקובל בתקשורת אלחוטית.



Time-division multiplexing – שיטה שבה רק אחד מקבל את ההודעה מהערוץ.

Statistical multiplexing – נניח יש לנו 3 תחנות שמעבירות 100 פקטות במקסימום. אבל בממוצע המידע שעובר בערוץ הוא 20 פקטות. לכן, כדי לחסוך את השימוש במשאבים אנחנו נקצה את גודל הערוץ קצת מעל הממוצע כך שסיפיק לרוב המקרים.

מחלקים את האינפורמציה לחלקים קטנים הנקראים פקטות (בעברית: מנות), כל פקטה עוברת דרך ערוץ השידור, לא בהכרח לפי הסדר ולא רק אינפורמציה אחת. מס' פקטות ממקורות שונים עוברות על אותו ערוץ.

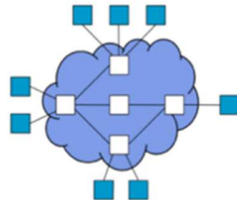
Networks issues: switching, scheduling

מיתוג switching

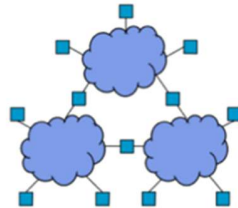
מתג הוא פשוט של הרשת, ביאילו קיפלו אותה ושמו אותה בתוך קופסה.

יש 2 סוגי חיבורים:

1. **Switched networks** – חיבור של תחנות לרשת של מתגים.



2. **חיבור בין-רשתות Internetworks** – כשהמציאו את האינטרנט הרעיון שלו היה לחבר 2 רשתות שונות קיימות (למשל רשת של בנק) ולקבל רשת מאוחדת.



סוגי מיתוג



1. **מיתוג מעגלים circuit switching** – המיתוג הזה עובד באופן דומה למרכזית שיחות. בעבר כשרצו לבצע שיחת טלפון היו מתקשרים למרכזיה ומבקשים לשוחח עם אדם מסוים. המרכזית בודקת אם האדם יכול לקבל את השיחה ובמידה וכן מחברת את שני האנשים.
עבור מיתוג זה יש:

- מנגנון של setup
 - משאבים שמיועדים לחיבור
 - Tear down – מנגנון שמשחרר את המשאבים
- זה בעצם מנגנון של סינון שאומר האם אתה מקבל את המשאבים לקיום השיחה או לא.
מקבלים קצב שידור קבוע constant bit rate.

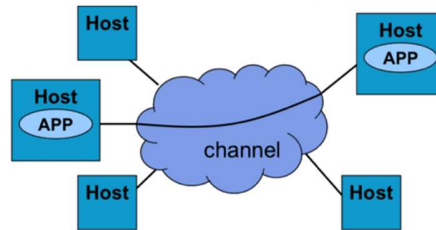
2. **מיתוג פקטות packet switching** – במיתוג זה לא מייצרים קו יעודי בין המקור ליעד אלא מחלקים את האינפורמציה לפקטות ומעבירים אותם על ערוץ אחד. בצד המקבל נוצרת אשליה של זרם רציף כמו במיתוג מעגלים.
חסרון: על אותו ערוץ יכולות להיות מחוברות מס' תחנות ולכן איכות התקשורת תלויה באחרים. אם הרשת תופצץ בהרבה פקטות זה יעכב את התנועה. במיתוג מעגלים זה לא קורה כי הקו הוא שלנו בלבד.

Scheduling – מכיוון שלצומת יש כניסות ויציאות זה עלול לגרום לפקק (congestion) ולכן עולה דילמה, מי נוסע עכשיו? במציאות למשל מכונית בכביש תיסע באותו הכיוון ובאותו הסדר בכביש. ברשת זה קצת אחרת ומנוגד ללוגיקה, כי לא אכפת לנו איך הפקטה תגיע ובאיזה סדר, העיקר שתגיע.
יש כל מיני סוגים של מנגנוני תזמון כמו round-robin, FIFO וכו'.

פונקציונליות – זה אומר איזה פונקציות תקשורת צריך לתת למשתמש. המשתמשים צריך ממשק פשוט ונוח והוא לא אמור לדעת את הפרטים על ניהול הרשת. במידה והמידע על ניהול הרשת דולף למשתמש – זו אי-הצלחה של הרשת.

דוגמה לשירות תקשורת: ערוץ

בתמונה מטה אנו רואים מחשבים שמחוברים לענן.



האפליקציה היא רק קוראת את המידע, כמו לקרוא מידע socket (socket הוא כמו קובץ) והיא לא צריכה לטפל בפקטות שעוברות ובאיך שהן עוברות. למשל אם יש לנו דפדפן אינטרנט, כשאנחנו מקליקים על המסך אנחנו רק רוצים שזה יציג לנו את מה שביקשנו. זו איזו שהיא אבסטרקציה של פרוטוקול TCP.

הבניה של האבסטרקציה הזאת היא אתגר הנדסי. הלוגיקה הזו של האבסטרקציה יושבת בתוך הרשת במיתוג המעגלי. באינטרנט עשו את האבסטרקציה הזאת בקצוות (host-ים) ואנחנו סובלים מזה עד היום. זאת משום שלא התכוונו שהאינטרנט יהיה בשימוש כלל עולמי רחב כל כך.

הרשת מועדת לתקלות כי יש בה המון רכיבים ואנחנו רוצים להסתיר את התקלות האלו מהמשתמש ע"י יתירות. התקלות יכולות להתרחש ב:

- ביטים – הביטים יכולים להתהפך או להקצץ.
- קווים – הקו יכול להקרע למשל ע"י טרקטור שנסע על קבל תקשורת.
- מעבדים – המעבד יכולים לקרוס בשל בעיות חומרה/תוכנה.
- התקפות עיונות – יכולה להתרחש תקלה בשל התקפה עוינת.

Network performance

ביצועים

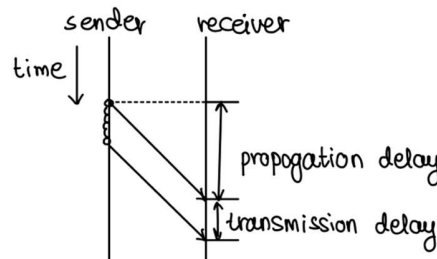
אנחנו חושבים על ערוץ תקשורת כמו צינור ולו 2 מימדים חשובים: אורך ורוחב.

1. אורך – latency – כמה זמן לוקח לביט אחד להגיע לצד השני. זה בד"כ $\frac{2}{3}$ מהירות האור.
2. רוחב סרט bandwidth – כמה ביטים אפשר להעביר בצינור ביחידת זמן. מודדים בביטים לשנייה בבסיס 10. זיכרון לעומת זאת מודדים מודדים בבתים בבסיס 2.



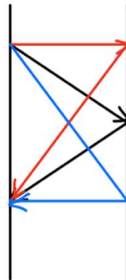
Latency

- **Propagation delay** – הזמן מהרגע שהודעה נשלחה עד שהתקבלה. זה המרחק חלקי המהירות.
- **Transmission delay** – הזמן מהרגע שהעברנו את הפקטה הראשונה עד לפקטה האחרונה. גודל הדאטא חלקי רוחב הסרט.



- **Queuing delay** – דילאי שנובע מהמעבר בין קו אחד לקו אחר.
- **Software processing overhead**

- **RTT (Round Trip Delay)** – הזמן שלוקח להודעה הכי קצרה לעבור ליעד ולחזור אלינו בחזרה. לא ניתן לדעת אם במקרה ההודעה הגיעה מהר מאוד והתעכבה בחזרה או להפך.



Layering and protocols

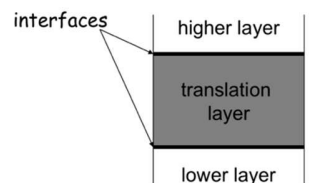
פרוטוקולים

כמו שאמרנו זו דרך לפרש ביטים. אבל באופן כללי יותר זו מערכת כללים לאיך לעשות תקשורת. הפרוטוקול מורכב מ:

- Syntax
 - סמנטיקה – משמעות ההודעה.
 - אומר איך לסגור את התקשורת.
 - מטפל בחבילות שמגיעות לא לפי הסדר.
- למשל דפדפן אינטרנט זה שרת שיועד לדבר בפרוטוקול http. ה client שולח בקשה וה server עונה בsockets של TCPn.

חלוקה לשכבות

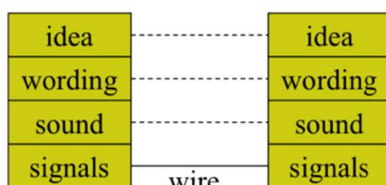
בהנדסה (ובחיים בכלל) נהוג לפרק בעיה גדולה לתת בעיות קטנות. עושים זאת ע"י חלוקה לשכבות.



השכבה התחתונה מספקת שירותים, השכבה העליונה דורשת דברים והשכבה האמצעית מספקת שירותים יותר מורכבים. לכן זה לא משנה לנו מה יש בשכבה האמצעית, מה שחשוב הוא במה אנחנו משתמשים ומה אנחנו מספקים ולכן יש חשיבות גבוהה לinterface. יתרון: מודלריות – אפשר להחליף את השכבות ואת סדרן. חסרון: ירידה בביצועים – יש תלות בין השכבות ולכן אי אפשר לייעל את המערכת כולה אלא רק את תתי החלקים שבה.

מחסנית שכבות Protocol stack

דוגמא – שיחת טלפון בטלפון קווי

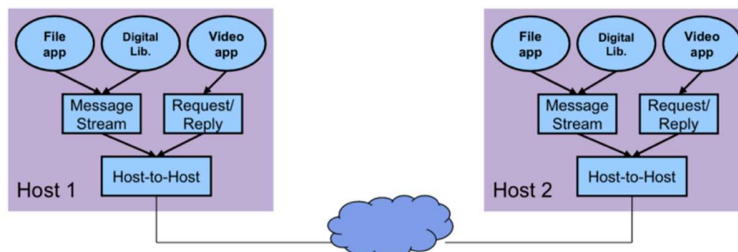


- אם למשל עולה במוחינו רעיון ללכת לסרט עם חבר שלנו משה.
- את הרעיון אנחנו מתרגמים לניסוח מילולי.
- את הניסוח המילולי אנחנו אומרים בקול בטלפון.
- הטלפון ממיר את הסאונד לסיגנלים חשמליים אשר עוברים בחוט לצד השני.
- משה בצד השני שומע את הסאונד בטלפון, מפרש את הסאונד למילים ומתרגם את המילים לרעיון.

כלומר התקשורת הפיזית מבוצעת ברמה הנמוכה ביותר אבל התקשורת היא קונספטואלית. לכל אחת מהשכבות יש חוקים משלה, למשל לשפה יש חוקים משלה או תקנים שקיימים בטלפון לתרגום הסאונד לסיגנלים החשמליים. כל שכבה מנצלת את המשאבים מהשכבה מתחתיה ומספקת שירותים לשכבה מעליה. זה נקרא מחסנית שכבות.

בתמונה למטה אנחנו רואים מבנה קצת יותר מפורט בעל שני סוגי תקשורת:

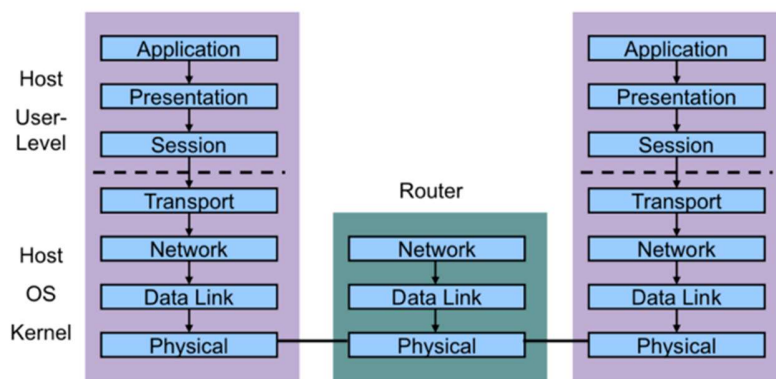
- Message stream
- Request/reply



הכנסה Encapsulation – המודול האמצעי מקבל הודעה מהאפליקציות מעלה וסוגר את ההודעה במעטפה. על המעטפה הוא רושם את היעד (זה נקרא header), למשל הודעה שמיועדת לmessage stream. בצד השני הוא מקבל את המעטפה, קורא את היעד ומעביר אותו לשם. המודול האמצעי פותח את המעטפה ומעביר את המידע שבפנים לאפליקציה מעלה. כעת נדון במבנה המפורט יותר למודל השכבות.

Network layer model

OSI Model



בין 2 host-ים אנחנו רואים ראוטר (זה לא הראוטר שיש לנו בבית).

היום השכבות של presentation וה-session נעלמו.

1. Physical layer – זה בעצם צינור של ביטים. תפקידה לייצג חיבור פיזי. השכבה הזאת אומרת איך מייצגים ביט, איך נקרא תקע/שקע של קבל רשת.

2. Data link – physical layer קשורה ישירות לשכבת הdata link. הוא לוקח את הphysical layer ומייצר ממנה תקשורת ברמה של פריימים ולא ברמה של ביטים. ישנו גם סימון לאיך פריימים מתחיל/נגמר. MAC (Medium Access Control) – תת-שכבה שאומרת איך ניגשים. MAC address – הכתובת של כרטיס הרשת. מורכב מ48 ביט. זו לא כתובת IP.